

桌面宠物（Desktop Pet）项目计划书 & 开发计划

版本 v1.0 • 2026-06-24 技术路线：Mac 端原生 Swift / SwiftUI + AppKit, 3D 模型形象，本地优先，后续扩展四端网络同步

1. 项目背景与愿景

打造一只常驻桌面的 3D 萌宠（参考形象：奶白色小猫 / 小狗，头顶嫩芽、薄荷绿围巾配铃铛）。她以透明无边框窗口悬浮在桌面最前层，眼睛实时跟随鼠标转动，用户可以戳一戳、拖动、喂食、抚摸与她互动，触发不同表情（开心、惊讶、眯眼笑等，对应原图六视图中的多种表情）。

产品分两大阶段演进：

- 1. 阶段一（本期）：Mac 端单机本地实现，把"养、逗、看"的核心体验跑通，零网络依赖。
- 2. 阶段二（后续）：引入网络能力，实现 Mac / iPhone / Android / Windows 四端宠物状态同步——同一只宠物的心情、饥饿度、成长值、装扮在多设备间一致。

目标用户当前聚焦在拥有 Mac + Android 的开发者本人，先自用打磨，再逐步对外。

2. 产品范围（Scope）

2.1 阶段一 • Mac 本地 MVP 必做功能

模块	功能点	说明
桌面呈现	透明无边框窗口、始终置顶、可跨所有 Space、点击穿透（非宠物区域）	宠物像贴在桌面上
3D 渲染	加载 3D 模型，待机呼吸动画、自然摆动	SceneKit / RealityKit
眼睛追踪	眼球 / 头部随全局鼠标位置转动	全局鼠标监听 + 朝向插值
互动	鼠标拖动移动位置、点击 / 戳触发表情、双击播放特殊动作	命中检测
表情状态机	开心 / 惊讶 / 眯眼笑 / 待机 等表情切换	对应原图表情集
桌宠数值	饥饿、心情、活力等本地养成数值，随时间衰减	本地持久化
菜单栏入口	状态栏图标：显示/隐藏、喂食、设置、退出	NSStatusItem
设置	大小、位置锁定、开机自启、是否点击穿透	本地配置
持久化	关闭后保留位置、数值、装扮	本地存储

2.2 阶段一 不做（明确边界）

- 不做任何网络请求 / 账号系统。
- 不做 iPhone / Android / Windows 端（仅做架构预留）。
- 不做内购、皮肤商城。
- 语音 / 桌面感知（读屏、提醒）放入待定增强项。

4.1 透明、置顶、点击穿透的桌宠窗口

用 `NSWindow`（或 `NSPanel`）而非默认 SwiftUI `Window`：

- `styleMask = .borderless`：无标题栏边框。
- `backgroundColor = .clear`、`isOpaque = false`、`hasShadow = false`：透明背景。
- `level = .floating`（或 `.statusBar` / `.screenSaver` 视置顶强度而定）：浮在普通窗口之上。
- `collectionBehavior = [.canJoinAllSpaces, .stationary, .fullScreenAuxiliary]`：跨所有桌面、全屏应用上方也能显示。
- **点击穿透**：默认 `ignoresMouseEvents = true`，让点击落到下方应用；仅当鼠标进入宠物实际像素/包围盒时，动态切回 `false` 接收互动。用全局鼠标位置 + 命中检测实现“只有点到宠物才拦截”。

4.2 眼睛 / 头部跟随鼠标

- 用 `NSEvent.addGlobalMonitorForEvents(matching: .mouseMoved)` 监听**全屏范围**鼠标移动（不在窗口内也能拿到坐标）。
- 把屏幕鼠标坐标换算到宠物模型本地坐标，求出朝向向量。
- 在 SceneKit 中对眼球节点 / 头部节点做 `SCNAction` 或每帧 `eulerAngles` 插值（带阻尼/缓动），避免生硬；设置最大转动角度，超出范围只动眼睛不动头，更自然。
- 性能：节流到 30–60fps，鼠标静止时停止更新。

隐私提示：全局鼠标监听在 macOS 上需要**辅助功能（Accessibility）权限**。首次启动应引导用户在「系统设置 → 隐私与安全性 → 辅助功能」授权，并在未授权时降级为“只在窗口范围内追踪”。

4.3 3D 模型与动画

- 美术产出 USDZ / glTF 模型 + 骨骼动画（待机呼吸、眨眼、开心跳、惊讶、眯眼笑）。
- SceneKit 加载为 `SCNScene`，嵌入 `SCNView`（放进透明 `NSWindow` 的 `contentView`）。
- 表情/动作用 `SCNAnimationPlayer` 管理，按状态机切换并做交叉淡入淡出（blend）。
- 待机时叠加细微随机动作（看向别处、甩尾、打盹）增加生命感。

4.4 互动与状态机

- 命中检测：`SCNView.hitTest` 判断点击是否落在宠物上。
- 交互意图：单击=戳（惊讶/害羞）、长按/抚摸=开心、拖动=移动窗口、双击=特殊动作、菜单喂食=回复饥饿。
- 状态机：`Idle` / `Happy` / `Surprised` / `Eating` / `Dragged` / `Sleepy`，事件驱动转移，每个状态绑定动画 + 数值变化。

4.5 养成数值与持久化

- `PetState`：hunger、mood、energy、growth、lastUpdate。
- 用 `Timer` 定时衰减（如每分钟饥饿 +x），App 重启时按 `lastUpdate` 补算离线流逝。
- SwiftData 存档；窗口位置等存 `UserDefaults`。
- **数据模型从一开始就用 `Codable`**，为阶段二上云做准备。

4.6 系统集成

- `NSStatusItem` 菜单栏图标 + 菜单。
- 开机自启：`SMAppService`（macOS 13+）。
- 多屏支持：记录所在屏幕。
- 打包：开发者签名 + 公证（Notarization），便于分发。

5. 阶段一开发计划（Mac 本地 MVP）

按单人/小团队节奏估算，单位为"周"，可按实际并行压缩。

Sprint 0 • 准备（约 0.5 周）

- 建 Xcode 工程、Git 仓库、分层目录骨架（Presentation / Interaction / Domain / Data）。
- 定义 `PetState`、`Renderer` 协议、`Repository` 协议（先空实现）。
- 准备一个临时 3D 占位模型（先用 Apple 示例 USDZ 跑通管线）。
- **验收**：空窗口能显示一个 3D 立方体/占位模型。

Sprint 1 • 桌宠窗口（约 1 周）—— 最高风险，先攻

- 实现透明、无边框、置顶、跨 Space 的 `NSWindow`。
- 实现点击穿透 + 命中检测切换。
- 拖动移动 + 位置持久化。
- **验收**：占位模型悬浮桌面、可拖动、非宠物区域点击穿透到下层应用。

Sprint 2 • 3D 渲染与眼睛追踪（约 1.5 周）—— 核心体验

- 接入正式 3D 模型 + 待机呼吸动画。
- 全局鼠标监听 + 辅助功能权限引导。
- 眼球/头部跟随鼠标（带缓动与角度限制）。
- **验收**：宠物眼睛丝滑跟随鼠标，待机有生命感。

Sprint 3 • 互动与表情状态机（约 1.5 周）

- 状态机 + 各表情/动作动画切换（对应原图表情集）。
- 单击/长按/双击/拖动交互映射。
- **验收**：戳她会惊讶、抚摸会开心笑，动作切换自然。

Sprint 4 • 养成数值与系统集成（约 1 周）

- 数值衰减 + 离线补算 + `SwiftData` 存档。
- 菜单栏图标、喂食、设置面板、开机自启。
- **验收**：喂食/心情生效，重启后状态保留。

Sprint 5 • 打磨、签名、发布（约 1 周）

- 性能优化（CPU/GPU/内存占用、空闲降帧）。
- 边界处理（多屏、权限拒绝降级、退出清理）。
- 签名 + 公证 + 打 DMG。
- **验收**：长时间运行稳定、占用低、可分发安装。

总计约 6.5~7 周（单人）。建议把 Sprint 1（桌宠窗口）当作技术验证 spike 优先攻克，它决定整个产品可行性。

关键风险与应对

风险	影响	应对
透明置顶 + 点击穿透在新系统/全屏下表现不稳	高	Sprint 1 先做 spike 验证；多 <code>collectionBehavior</code> 组合测试

风险	影响	应对
全局鼠标需辅助功能权限，用户可能拒绝	中	提供降级模式（窗口内追踪）+ 清晰授权引导
3D 实时渲染常驻导致耗电/占用高	中	空闲降帧、鼠标静止停更、Metal 优化
美术 3D 资源产出周期	中	先用占位模型跑通管线，资源到位再替换

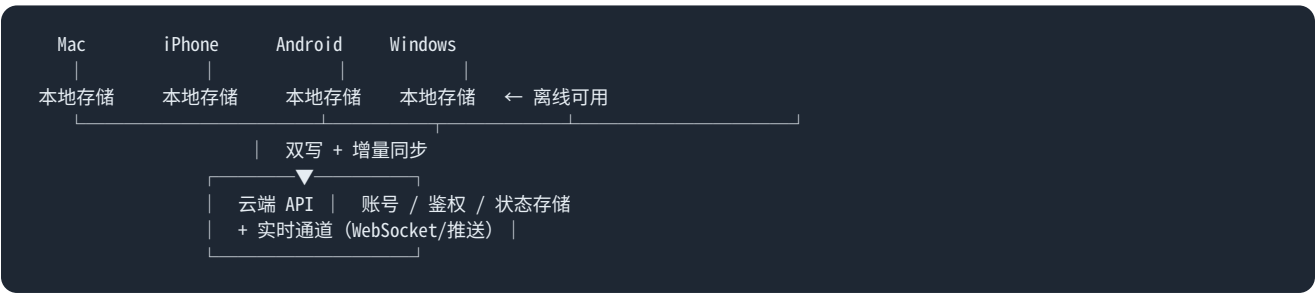
6. 阶段二 · 网络同步与四端扩展（后续规划）

本期不实现，仅做架构预留，确保阶段一的 Domain/Data 设计不阻塞未来。

6.1 目标

同一只宠物在 Mac / iPhone / Android / Windows 间状态一致：心情、饥饿、成长、装扮、位置偏好等近实时同步；支持离线编辑、上线合并。

6.2 同步架构（建议）



- **数据协议统一**：定义跨端 JSON / Protobuf 的 `PetState` schema（阶段一的 `Codable` 模型直接演进）。
- **同步策略**：本地优先 + 增量上传；冲突用「时间戳 + 字段级合并」或 CRDT（数值类适合 last-write-wins，成长值适合累加合并）。
- **实时性**：WebSocket 长连接做在线推送；离线靠 APNs（iOS/Mac） / FCM（Android）唤醒。
- **账号**：Sign in with Apple + 邮箱，跨端登录同一宠物。
- **后端**：可选 Firebase / Supabase 快速起步，或自建（Swift on Server / Go + Postgres + Redis）。

6.3 四端技术对照

端	渲染/窗口	复用
macOS	SceneKit + 透明 NSWindow	本期产出，Domain 全复用
iOS	SceneKit/RealityKit，App 内宠物（无桌面悬浮）	复用 Swift Domain 60%+
Android	SceneView/Filament/Unity + 悬浮窗(WindowManager)	重写渲染层，复用数据协议
Windows	WinUI/WPF 透明窗 + Win3D / 或 Unity 导出	重写渲染层，复用数据协议

若阶段二希望大幅降低多端渲染重复工作，可评估把**渲染层统一迁移到 Unity/Godot**，Swift 仅保留服务端与数据层——这是一个需要在阶段二专门决策的权衡点。

7. 里程碑总览

里程碑	内容	预期
M1	Mac 桌宠窗口 spike 通过（透明/置顶/穿透/拖动）	第 1.5 周
M2	眼睛跟随 + 3D 待机动画	第 3 周
M3	互动 + 表情状态机	第 4.5 周
M4	养成数值 + 系统集成	第 5.5 周
M5	Mac 本地 MVP 发布 (v1.0)	第 7 周
M6	后端 + 数据协议 + Mac 接入同步	阶段二
M7	iPhone 端上线	阶段二
M8	Android 端上线	阶段二
M9	Windows 端 + 四端打通	阶段二

8. 下一步建议

1. 先搭工程骨架 + Sprint 1 的透明置顶窗口 spike——这是整个项目可行性的命门，建议第一周就验证。
2. 并行准备 3D 美术资源（按原图风格输出 USDZ + 表情/动作动画清单）。
3. 数据模型一律 Codable 化，为阶段二同步铺路。

如果需要，我可以接着帮你： - 生成 **Xcode 工程初始骨架代码**（透明置顶窗口 + SceneKit 占位模型 + 眼睛追踪的可运行 demo）； - 输出 **3D 美术需求清单 / 动画状态表**； - 把本计划书导出为 **Word / PDF** 版本。